

The Prompt Ladder: Single-Layer Iteration (Week 2)

Track: General AI Fluency | Phase: Foundations | Author: Harshit Kudhial (@harshitttt077) | Date: September 2026

1. The Six Runs: Side-by-Side Progression & Observable Differences

Run / Version	Single Layer Added	What Actually Improved in Output	What Still Failed (Next Bottleneck)
Run 0 (Baseline)	None (Weak Baseline) "Help me find which pages to refresh..."	Baseline reference point.	Hallucinated columns (bounce_rate, pageviews); gave generic SEO blogging advice.
Run 1	Real Context & Schema Injected exact 44-col CSV path and pre-decision columns.	Stopped hallucinating columns; correctly ingested local CSV and real fields.	Outputted an unranked binary filter dumping 16,180 rows (no capacity concept).
Run 2 (Failure)	Clear Operational Goal Top-50 queue, Precision@50 capacity constraint.	Shifted from massive unranked dump to an ordered, scored top-50 queue.	Catastrophic Leakage! Injected trend_pct into features; cheated with false 1.000 score.
Run 3	Negative Constraints Strict leakage ban on trend_pct; client-holdout split.	False 1.000 collapsed to honest 0.680 holdout Precision@50; no client overlap.	Raw terminal array output; no human reason codes or executive summary.
Run 4	Specified Output Format Markdown comparison table + top-10 reason codes.	Produced executive Markdown benchmark table and actionable attributed queue.	Code lacked defensive verification; no sanity checks on nulls or split leak.
Run 5 (Production)	Defensive Assertions Mandatory assertions for zero leakage, split integrity, 2x lift.	Self-verifying enterprise pipeline; automatically halts if leakage or split fails.	None within scope. Certified ready for production deployment.

2. Deep-Dive: The Honest 'This Made It Worse' Moment (Run 2 -> Run 3)

In **Run 2**, adding the goal of 'maximizing Precision@50' created an unintended perverse incentive: the AI greedily included trend_pct in the feature matrix X. Because is_declining = (trend_direction == 'down') is computed directly from trend_pct < -0.10, the model achieved a **perfect Precision@50 of 1.0000**. The output looked phenomenal on paper but was completely broken in reality—it learned to read the answer key rather than predicting real-world decay. Adding **Negative Constraints (Run 3)** that explicitly forbade target-derived fields collapsed the score to an honest **0.680 holdout Precision@50**.

3. Final Reusable Master Prompt (Production Template)

[PROMPT TEMPLATE FOR ML RANKING]

You are an expert ML Systems Engineer specializing in search performance analytics.

- CONTEXT: Dataset at {DATASET_PATH} with {ROW_COUNT} rows. Pre-decision features: content_age_days, days_since_last_update, impressions_90d, ctr, avg_position, word_count, engagement_rate. Target: is_declining = (trend_direction == 'down').
- GOAL: Output an ordered review queue of the top {CAPACITY_K} pages in need of refresh. Evaluate Precision@{CAPACITY_K} on unseen clients vs baseline.
- CONSTRAINTS: ZERO LEAKAGE (never use trend_pct/trend_direction in features); GROUPED CLIENT SPLIT (20% client holdout); NO PII.
- VERIFICATION: Assert zero client overlap; assert no label columns in features; assert exactly {CAPACITY_K} rows with zero null scores; assert >=2.0x lift.
- OUTPUT: Complete Python script + Markdown benchmark table + Top 10 candidate table with explicit reason codes.